

基于图论的校园最优游览路线规划——一个广义中国邮递员问题的求解

引言

随着毕业季的到来，一名中国科学技术大学高新校区的学生计划在离校前进行一次有特殊意义的校园告别之旅。这次旅程并非随意的漫步，而是有着明确的目标：走遍校园内的七条主要道路（东西南北卓越路、华罗庚路、赵九章路、赵忠尧路、夏培肃路、永恒路、思佩桥），并在全部二十个标志性建筑前拍照留念。该生希望在满足所有这些任务的前提下，规划出一条连续的、总路程最短的路线，以最高效的方式完成这次充满仪式感的校园巡礼。

该问题本质上是运筹学和图论领域一个经典且具有挑战性的问题，被称为***“广义中国邮递员问题” (Generalized Chinese Postman Problem, GCPP)**，或混合邮递员问题。它混合了两种约束：既要遍历一组特定的“边”（道路），又要访问一组特定的“点”（地标）。此类问题在现实世界中具有广泛的应用价值，是许多现代物流与城市服务系统优化的核心，例如城市邮政的高效投递、市政垃圾回收车的最优路线、冬季道路的清雪作业、以及电力或通信管线的巡检等。这些应用场景的共同目标都是在满足一系列服务需求下，实现运营成本（如时间、距离、燃料消耗）的最小化。

为科学地解决此问题，本报告首先将高新校区的平面地图抽象为一个加权无向图 $G=(V, E)$ 。其中，图的**节点 (V)** 代表校园内的各地标建筑、大门及道路交叉口；**边 (E)** 代表连接这些地点的实际路径，其**权重 (w)** 则被设定为对应路径的长度，以此代表旅行时间或成本。任务相应地被精确转化为图论中的约束条件：一个必须访问的**节点集合 (V_req)** 和一个必须遍历的**边集合 (E_req)**。

本研究的求解过程完全基于Python编程语言，并深度借助了强大的图论计算库 **NetworkX** 进行建模、分析与计算，同时使用 **Matplotlib** 库对各阶段的结果进行可视化展示。报告将遵循以下技术路线，系统地阐述问题的解决全过程：

- 数字化建模**：将校园地图数据化，创建精确的图模型。
- 约束识别**：在图中识别出必需访问的节点和必需遍历的边。
- 网络连通**：通过计算最短路径，将所有离散的必需组件连接成一个单一的连通网络。
- 邮递员求解**：应用中国邮递员问题的核心算法，通过最小权重完美匹配，计算出为保证路线“一笔画”走完所必需的最优重复路径。
- 路线构建与分析**：整合所有路径，构建最终的欧拉回路（即最优游览路线），并对总用时和具体路线序列进行分析。

通过这一系列步骤，本报告旨在为这个具体而有趣的校园游览问题，提供一个完整的、可复现的、算法驱动的解决方案。

第一部分：校园图谱的数字化建模与数据准备

1.1 建模目标

所有复杂的路线优化问题都始于一个精确的现实世界模型。本部分的核心目标是将中国科学技术大学高新校区的二维平面地图，转化为一个计算机可以识别、计算和分析的 **加权无向图 (Weighted Undirected Graph)** 。这个数字化的图谱是后续所有算法分析、路径计算和路线规划的基石，其准确性直接决定了最终结果的有效性。

1.2 建模过程

我们将校园抽象为图 $G=(V, E)$ ，其建模过程主要包括对节点 (V) 和边 (E) 的定义与量化。

1.2.1 节点的抽象与定义 (Node Abstraction)

图中的节点代表了校园中所有关键的地理位置。为了精确地描述路网结构，我们不仅将学生必须访问的 **20个地标建筑 (B1-B20)** 和 **4个校门 (G_N, G_S, G_East, G_W)** 定义为节点，更关键的是，我们将所有主要的道路交叉口 (**I1-I7**) 也抽象为节点。

相应的节点在实际地图中对应关系如下图:



引入交叉口节点至关重要，因为它们是路网的拓扑枢纽，是路线产生分叉和选择的决策点。若忽略交叉口，则无法准确表达路径之间的连通关系。

为便于后续计算和可视化，我们为每一个节点都分配了一个唯一的ID，并根据其在地图上的相对位置，在一个二维笛卡尔坐标系中估算了其坐标 (x, y) 。(整个地图的单位是 1000×1000)。

具体的节点坐标可以查看附件 `graph_creation.py`。

1.2.2 边的抽象与定义 (Edge Abstraction)

图中的边代表了连接各个节点的实际校园道路或小径。在我们的模型中，每一条边都具有以下两个核心属性：

- **名称 (Name)**：如“North Zhuoyue Road”，用于识别该路段所属的道路名称。
- **权重 (Weight)**：边的权重是路线优化的核心依据，它代表了通过该路段的“成本”。在本模型中，我们采用两个节点坐标之间的**欧几里得距离**来计算权重，以此作为行程时间或距离的量化指标。计算公式如下：

$$w(u, v) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2}$$

其中, (x_u, y_u) 和 (x_v, y_v) 分别是边所连接的两个节点 u 和 v 的坐标。

1.3 建模实现与结果

我们使用 Python 的 `networkx` 库来构建图模型。通过编程，我们将定义的节点及其属性（ID, 名称, 坐标）和边及其属性（名称, 权重）加载到 `networkx.Graph` 对象中。

经过最终的数据校正与整合，我们构建的校园图谱包含 **31个节点** 和 **44条边**。为了验证模型的准确性，我们利用 `matplotlib` 库对生成的图谱进行了可视化，如下**图1**所示。

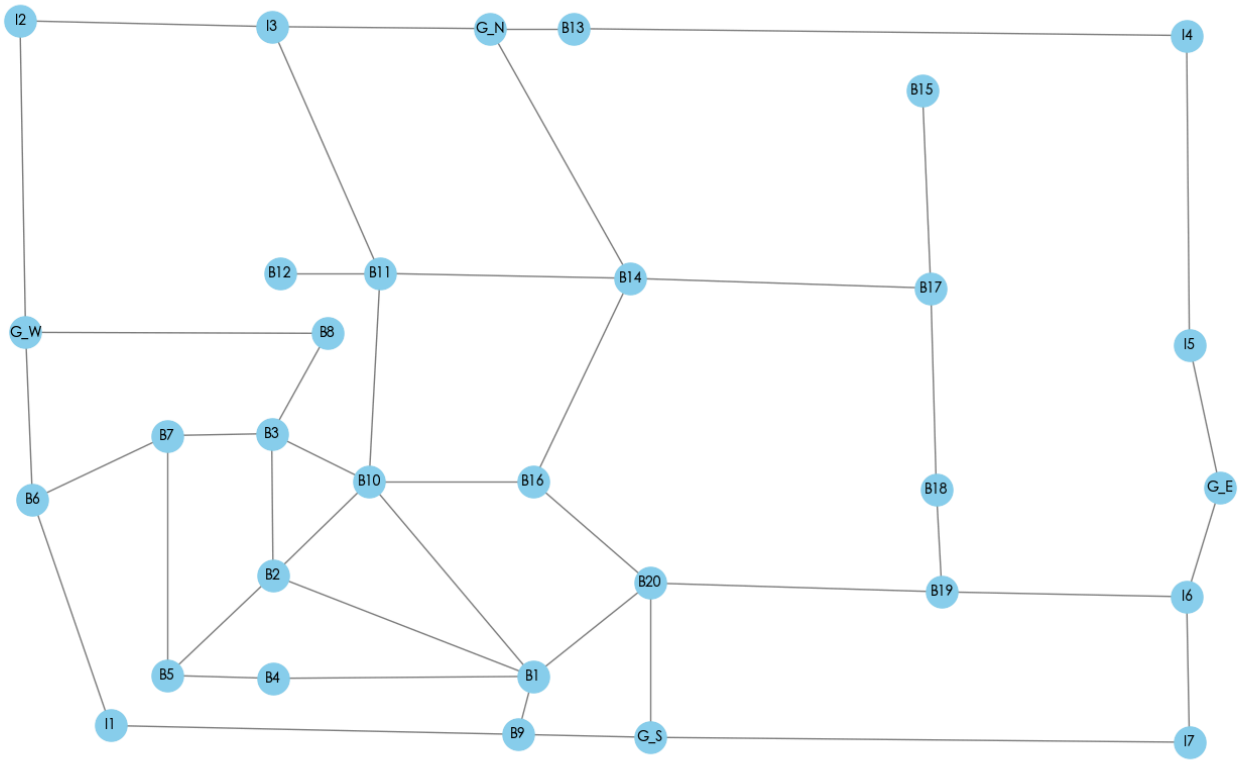


图1 直观地展示了校园路网的拓扑结构。图中清晰地标示了所有地标、大门和交叉口的位置及其连接关系。通过将此图与原始地图进行比对，我们可以确认模型在宏观结构上准确地复现了真实的校园环境。这个经过验证的数字化图谱，为我们接下来识别必需组件、分析网络连通性以及最终求解最优路线提供了可靠的数据基础。

第二部分：必需游览组件的识别与定义

2.1 约束条件的数学化

在完成校园的数字化建模后，下一步是**将抽象的游览任务转化为精确的、可供算法处理的图论约束**。此步骤的目标是将“走遍七条路”和“打卡二十个地标”这两个定性要求，精确地映射为图 G 中的一组必需节点 V_{req} 和一组必需边 E_{req} 。

2.2 必需节点 (V_{req}) 的定义

学生的任务要求在20个标志性建筑前拍照留念。在我们的图模型中，这20个地标建筑（B1 至 B20）被直接定义为**必需访问节点集**，记为 V_{req} 。

$$V_{req} = \{B1, B2, ..., B20\}$$

这个集合包含了20个节点。最终规划的路线必须经过（或至少“接触”到）此集合中的每一个节点。

第三部分：必需网络连通性的构建

3.1 连通性的必要性与挑战

在第二部分中，我们识别出的必需游览组件（红色部分）在校园中呈“孤岛”状分布。然而，一次有效的旅行必须是一条连续不断的路径。因此，在规划最终路线之前，我们必须首先解决**网络连通性**问题：即找到总长度最短的额外路径，将所有这些分散的“必需岛屿”连接成一个单一、完整的连通网络。

从理论上讲，寻找连接多个点和边的绝对最短网络是一个著名的计算难题，称为***“斯坦纳树问题” (Steiner Tree Problem)**，属于NP-Hard问题，在复杂网络中难以求得最优解。因此，我们采用了一种高效且实用的启发式算法来解决此问题。

3.2 连接算法设计与实现

我们设计的连接策略可以被形象地比喻为“大陆与岛屿的桥梁建设”，具体实现步骤如下：

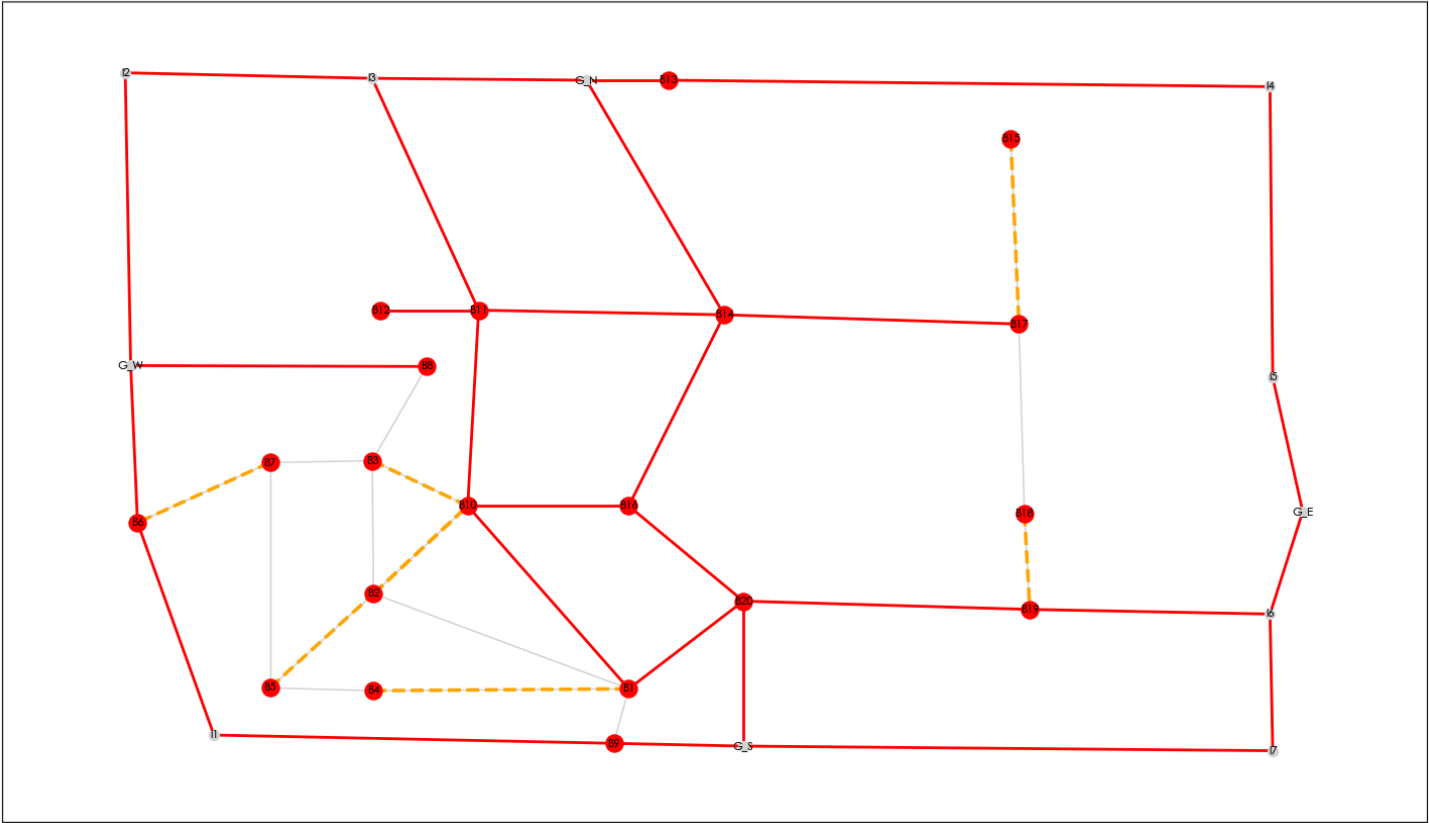
- 识别岛屿：**首先，我们创建一个仅包含所有必需节点 (V_{req}) 和必需边 (E_{req}) 的临时子图。然后，利用 networkx 的 `connected_components` 算法，精确地识别出该子图中所有互相独立的连通分量，即“岛屿”。
- 确定主大陆：**在所有识别出的“岛屿”中，我们将包含节点数量最多的那个定义为“主大陆”。
- 修建最短桥梁：**接着，算法会遍历每一个“小岛屿”。对于每一个小岛屿，它会在完整的校园图谱 G 中，计算并找出连接该岛屿上某一点到主大陆上某一点的、距离最短的一条路径。这条路径就是连接该岛屿的最优“桥梁”。
- 构建连通网络：**最后，我们将所有这些计算出的最短“桥梁”路径所包含的边，加入到我们的必需边集 (E_{req}) 中，形成一个扩展后的、完全连通的必需网络。

3.3 实现结果与可视化

通过执行上述算法，程序首先识别出了 **8个** 独立的必需组件。为了将它们全部连接起来，算法计算并增加了 **8个** 额外的连接路段。这些新增的路径确保了从任意一个必需的地点或道路出发，都可以通过一段连续的路径到达其他任何一个必需的地点或道路。

我们将这一结果进行了可视化，如下**图3**所示。

连接后的必需网络 (橙色为新增的连接路径)



在 图3 中，红色部分依然代表原始的必需组件。新增的橙色虚线部分，则代表了我们的算法自动计算出的、用于“缝合”所有红色孤岛的最短连接桥梁。

至此，一个由红色和橙色路径共同构成的、单一、完整的连通网络已经构建完成。这个网络囊括了所有学生必须走过的路段，为下一步——也是最核心的一步——求解“中国邮递员问题”提供了无懈可击的数据基础。

第四部分：最优“重复路线”的计算（中国邮递员问题求解）

4.1 问题的提出：欧拉回路与奇度节点

在第三部分中，我们已成功构建了一个连通的“必需网络”，确保了所有任务点之间路路相通。然而，连通不等于高效。根据图论中的**欧拉定理（Euler's Theorem）**，一个连通图能够被“一笔画”走完（即存在一条遍历所有边一次且仅一次的**欧拉回路**）的充要条件是，图中所有节点的**度（Degree）**均为偶数。

我们构建的必需网络（红色+橙色路径）很大概率会包含许多**奇度节点**——即连接了奇数条（1, 3, 5...）必需路径的交叉口。任何包含奇度节点的网络都无法一笔画走完，强行游览必然导致在这些节点处“卡住”，被迫进行低效的、随意的往返。

因此，本部分的核心任务是解决这个经典的***“中国邮递员问题”（**Chinese Postman Problem, CPP**）。其目标是，在必需网络的基础上，智能地增加一组总长度最短的额外“重复路线”（Deadheading

Paths)，用以“中和”掉所有的奇度节点，使整个网络升级为一个完美的欧拉图，从而保证最终游览路线的全局最优性。

4.2 求解算法：最小权重完美匹配

解决中国邮递员问题的标准算法精妙而高效，其步骤如下：

1. **识别奇度节点集**：首先，我们在步骤三构建的连通必需网络 `G_req_connected` 中，找出所有度为奇数的节点，构成集合 V_{odd} 。根据图论原理，任何图中奇度节点的数量必然是偶数。
2. **计算配对成本**：我们构建一个全新的、仅包含 V_{odd} 中节点的完全图。图中任意两个奇度节点 u 和 v 之间边的权重，被定义为它们在**完整的校园图谱 G** 中的最短路径长度。这个距离代表了将这两个奇度节点“中和”的最低成本。
3. **求解最小权重完美匹配**：这是算法的核心。我们在这个带权重的完全图上，应用**最小权重完美匹配 (Minimum-Weight Perfect Matching)** 算法。该算法能够找到一种配对方案，将所有奇度节点两两配对，并确保所有配对的“成本”（即最短路径距离）之和为全局最小。我们使用 `networkx` 库的 `min_weight_matching` 函数来实现这一功能。
4. **生成重复路线**：匹配方案给出了需要被连接的奇度节点对。我们根据这个方案，在完整的校园图谱 G 中找出每一对节点之间的实际最短路径。这些最短路径的集合，就是我们所求的最优“重复路线” `deadhead_edges`。

4.3 实现结果与可视化

算法执行后，在必需网络中总共识别出了 **18个** 奇度节点：

```
['G_N', 'I3', 'G_W', 'B6', 'B8', 'I6',  
'G_S', 'B1', 'B10', 'B16', 'B12', 'B19', 'B3', 'B4',  
'B5', 'B7', 'B15', 'B18']
```

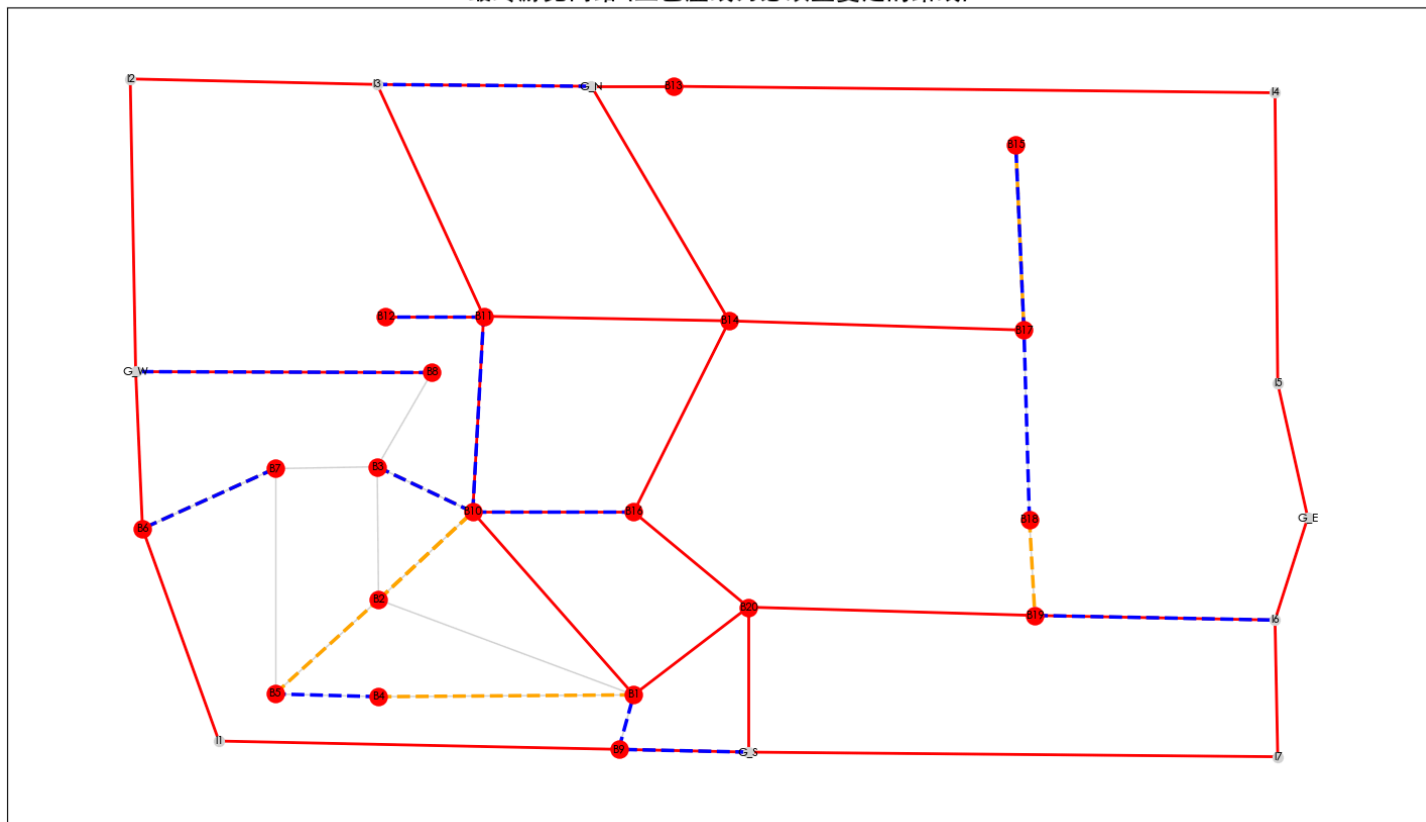
经过最小权重完美匹配计算，我们得到了一组连接这18个节点的最优“重复路线”，其总长度为 **1791.70** 个单位。

```
('B10', 'B16'), ('B18', 'B15'), ('B8', 'G_W'),  
( 'B3', 'B12'), ('B4', 'B5'), ('G_S', 'B1'),  
( 'B19', 'I6'), ('B7', 'B6'), ('G_N', 'I3')
```

这意味着，为了最高效地完成所有任务，学生必须额外（重复）行走这段距离。

我们将这一结果叠加在之前的图上，得到最终的游览网络构成**图4**。

最终游览网络 (蓝色虚线为必须重复走的路线)



在 图4 中，蓝色虚线代表了算法计算出的、必须重复走一遍的路线。这些路线巧妙地连接了网络中的“断点”，起到了“消除”奇度节点的关键作用。

至此，由必需路径（红色）、连接路径（橙色）和重复路径（蓝色）共同组成的网络，已经是一个完美的欧拉图。网络中的每一个节点都连接了偶数条路径，保证了存在一条可以顺畅走完所有任务的、总路程最短的闭合回路。我们已经为规划最终的路线序列，铺平了所有道路。

第五部分：最终游览路线的构建与分析

5.1 欧拉回路的生成方法

经过前面四个部分的分析与计算，我们已经构建了一个完美的欧拉图。这个图囊括了所有必需、连接和重复的路径，并确保每个节点的度均为偶数。本部分的目标就是从这个理论上的图中，提取出一条实际可执行的、连续的步行路线，即欧拉回路 (Eulerian Circuit)。

为了精确地构建这条回路，我们采用了一种能够处理重复路径的 `networkx` 数据结构——多重图 (MultiGraph)。这是因为在最优路线中，某一段物理路径可能需要被行走多次（例如，某条路既是必需走的，又恰好是最优的重复路线的一部分）。多重图允许两个节点之间存在多条边，从而能完整地记录每一次行走。

我们将所有三类路径（必需、连接、重复）的边集合并，加载到一个 `MultiGraph` 对象中。然后，指定一个出发点（例如，交通便利的南门 `G_S`），调用 `networkx` 库的 `eulerian_circuit` 函数。该函数能

够自动地、智能地沿着图中的边进行遍历，生成一个边的序列，这个序列就是我们所求的最优游览路线。

5.2 最终结果分析

5.2.1 定量分析

通过对最终欧拉图的所有边的权重进行求和，我们得到了完成本次校园游览任务所需的最短总路程。

- 最短游览总路程：**8772.74 单位**

这个总路程由三部分构成：

- 必需路径长度**：所有必需道路和必需地标连接路径的总长。这是完成任务所必须付出的基础成本。
- 连接路径长度**：为连接所有“孤岛”而增加的“桥梁”路径的总长。
- 重复路径长度**：为“中和”所有奇度节点而增加的“回头路”的总长，即**1791.70**单位。这部分是为保证路线效率和连通性而付出的、不可避免的额外成本。

最终的总路程是这三部分长度之和，代表了学生完成所有目标所需付出的最小代价。

5.2.2 路线序列展示

`eulerian_circuit` 函数不仅给出了总长度，更生成了具体的行走步骤。以下是从南门出发的最优游览路线的前15个步骤示例：

步骤	前往地点名称	地点ID
1 (起点)	南门	G_S
2	3号学科楼	B20
3	2号学科楼	B19
4	交叉口	I6
5	2号学科楼	B19
6	信息科学技术大楼	B18
7	1号学科楼	B17
8	行政服务中心	B15
9	1号学科楼	B17
10	教工公寓	B14

步骤	前往地点名称	地点ID
11	图书教育中心	B16
12	学生食堂	B10
13	3号学生公寓	B3
14	2号学生公寓	B2
15	学生食堂	B10
16	体育场	B11
17	体育馆	B12
18	体育场	B11
19	学生食堂	B10
20	2号学生公寓	B2
...	...	...

这份完整的节点序列清单，就是提供给学生的、傻瓜式的、最优的“导航指南”。

5.3 结果可视化

最终的游览路线构成如下图5所示，它全面总结了我们整个项目的计算结果。

最终游览路线构成图 (红:必需, 橙:连接, 蓝:重复)

